

Dhirubhai Ambani University

(Formerly known as DA-IICT)

Topic: File Handling in Python

Dr. Ankit Vijayvargiya

Assistant Professor

Room No. 4205, Faculty Block 4

Email: `ankit_Vijayvargiya[at]dau.ac.in`

File Handling

- File Handling is used to store a program's output in a file for later use.
- Data files are files that store data pertaining to a specific application for later use.
- The three types of files are:
 - Text file
 - Binary file
 - CSV file

File Handling

Text Files:

- A text file stores information in ASCII or Unicode characters.
- In a text file, each line of text is terminated with a special character known as EOL (End of Line).
- By default, this EOL character is the newline character (“\n”).

Character → ASCII number → Binary → Stored in file

Character → Unicode number → Binary → Stored in file

File Handling

Text File:

- To work with a file, the first step is to open the file. We may open the file for the following purposes:
 - Reading data from a file
 - Writing data to a file
 - Appending data to a file
- The following file-opening modes can be used to open a file.
 - r : read mode
 - w : write mode
 - a : append mode
 - x : create mode
- Syntax:
 - with open("file_name",mode) as fileobj:
.....
.....
 - fileobj= open("file_name",mode)

File Handling

File opening modes:



Read mode (r):

- Opens file for reading
- Error if file does not exist
- Default mode

Write mode (w):

- Opens file for writing
- Creates a file if it doesn't exist
- Overwrites existing content

Append mode (a):

- Adds data at the end
- Creates a file if it doesn't exist
- Does NOT delete old content

Create mode (x):

- Creates a new file
- Error if file already exists

• Examples:

```
file1=open("data.txt")
```

```
file1=open("data.txt","r")
```

```
file1=open("data.txt", "w")
```

```
file1=open("data.txt", "a")
```

```
file1=open("data.txt", "x")
```

If no mode is given, it is Read mode.

Opening a file in read mode.

Opening a file in write mode.

Opening a file in append mode.

Opening a file in create mode.

```
file1=open("D:\\mydata\\data.txt")
```

```
File1=open(r"D:\mydata\data.txt")
```

Closing a file:

```
file1.close()
```

File Handling

Reading a file content:

We have three methods to read a file's content (text file):

read()

readline()

readlines()

1. **read():** This method reads the entire file/given number of bytes from a file.

Syntax 1:

```
f1=open("abc.txt","r")
```

```
x=f1.read(5) # Here now x is a string with 5 characters from the file.
```

Syntax 2:

```
f1=open("abc.txt","r")
```

```
x=f1.read() # Here now x is a string containing everything from the file.
```

File Handling

Reading a file content:

We have three methods to read a file's content (text file):

2. **readline():** This method reads a line of input, or max n number of bytes if specified.

Syntax:

```
f1=open("abc.txt","r")  
x=f1.readline() # Here now x is a string, containing 1 line from the file.
```

3. **readlines():** This method reads all lines from the file, but in a list format.

Syntax:

```
f1=open("abc.txt","r")  
x=f1.readlines() # Here, now x is a list, containing all lines from the file.
```

Exercise

Create a file data.txt and write the following lines:

Hello Everyone,
I am Ram Sharma.
I am studying in Class 9.
I would like to learn Python programming.

File open

```
file1 = open("data.txt","r")
```

Read file using readlines()

```
file1 = open("data.txt","r")  
str1=file1.readlines()  
print(str1)  
file1.close()
```

```
['Hello Everyone,\n', 'I am Ram Sharma.\n', 'I am studying in Class 9.\n', 'I would like to learn Python programming.']
```

```
file1 = open("data.txt","r")  
str1=file1.readlines()  
for line in str1:  
    print(line.strip())
```

```
Hello Everyone,  
I am Ram Sharma.  
I am studying in Class 9.  
I would like to learn Python programming.
```

```
str1=file1.readlines()  
for line in str1:  
    print(line, end='')
```

Read file using read()

```
str1=file1.read(10)  
print(str1)
```

Hello Ever

```
str1=file1.read(10)  
print(str1)
```

yone,
I am

```
file1 = open("data.txt","r")  
str1=file1.read()  
print(str1)
```

Hello Everyone,
I am Ram Sharma.
I am studying in Class 9.
I would like to learn Python programming.

Read file using readline()

```
file1 = open("data.txt","r")  
str1=file1.readline()  
print(str1)
```

Hello Everyone,

```
str1=file1.readline()  
print(str1)
```

I am Ram Sharma.

```
file1 = open("data.txt","r")  
str1=""  
while str1:  
    str1=file1.readline()  
    print(str1,end='')  
file1.close()
```



File Handling

Writing data into files:

We have the following functions to write data into files:

- **write():** Writes a string to the file.

Syntax:

```
f1=open("abc.txt","w")  
x=f1.write(str1)
```

- **writelines():** Writes all strings in the list as lines to the file.

Syntax :

```
f1=open("abc.txt","w")  
x=f1.writelines(str1)
```

File Handling

Writing data into files:

```
file1 = open("data.text", "w")
```

```
str1=file1.read()  
print(str1)
```

```
-----  
UnsupportedOperation  
Cell In[29], line 1  
----> 1 str1=file1.read()  
      2 print(str1)
```

```
UnsupportedOperation: not readable
```

```
file1 = open("data.text", "w")  
for i in range(5):  
    name=input("Enter Name")  
    file1.write(name) #will simply write the content without adding any character  
file1.close()
```

```
Enter Name Ankit  
Enter Name Vijay  
Enter Name Vargiya  
Enter Name Ram  
Enter Name Shyam
```



```
1 AnkitVijayVargiyaRamShyam
```

```
: file1 = open("data.text", "w")  
for i in range(5):  
    name=input("Enter Name")  
    file1.write(name)  
    file1.write('\n')  
file1.close()
```

```
Enter Name Ankit  
Enter Name Vijay  
Enter Name Vargiya  
Enter Name Ram  
Enter Name Shyam
```



```
1 Ankit  
2 Vijay  
3 Vargiya  
4 Ram  
5 Shyam  
6
```

File Handling

Writing data into files:

```
: file1 = open("data.text","w")
list1=["Ram","Shyam", "Geeta"]
file1.writelines(list1)
file1.close()
```



1 RamShyamGeeta

```
: file1 = open("data.text","w")
list1=[]
for i in range(5):
    name=input("Enter Name")
    list1.append(name+'\n')
file1.writelines(list1)
file1.close()
```

Enter Name Ram
Enter Name Shyam
Enter Name Geeta
Enter Name Ankit
Enter Name Vijay



1 Ram
2 Shyam
3 Geeta
4 Ankit
5 Vijay
6

```
: file1 = open("data.text","w")
for i in range(3):
    roll=int(input("Enter Roll No. "))
    name=input("Enter Name")
    marks=float(input("Enter Marks"))
    rec=str(roll)+", "+name+", "+str(marks)+"\n"
    file1.write(rec)
file1.close()
```

Enter Roll No. 1
Enter Name Ram
Enter Marks 88.8
Enter Roll No. 2
Enter Name Shyam
Enter Marks 77.2
Enter Roll No. 3
Enter Name Geeta
Enter Marks 95.3

1 1,Ram,88.8
2 2,Shyam,77.2
3 3,Geeta,95.3
4

File Handling

Appending data into files:

```
: file1 = open("data.text","a")
  for i in range(3):
    roll=int(input("Enter Roll No. "))
    name=input("Enter Name")
    marks=float(input("Enter Marks"))
    rec=str(roll)+","+name+","+str(marks)+"\n"
    file1.write(rec)
  file1.close()
```

```
Enter Roll No. 4
Enter Name Ankit
Enter Marks 77
Enter Roll No. 5
Enter Name Vijay
Enter Marks 65
Enter Roll No. 6
Enter Name Vargiya
Enter Marks 93
```

```
1 1,Ram,88.8
2 2,Shyam,77.2
3 3,Geeta,95.3
4 4,Ankit,77.0
5 5,Vijay,65.0
6 6,Vargiya,93.0
7
```

File Handling

Flush function:

- The flush() method in Python is used to quickly send data from the computer's temporary storage, known as a buffer, to a file.

```
file1 = open("data.text","w")
for i in range(3):
    roll=int(input("Enter Roll No. "))
    name=input("Enter Name")
    marks=float(input("Enter Marks"))
    rec=str(roll)+","+name+","+str(marks)+"\n"
    file1.write(rec)
    file1.flush()
```

- Python automatically flushes the files when closing them, which means the flush() function is automatically called when the close() function is called.

File Handling

Binary File:

- The files that are present in the form of 0 or 1 are referred to as Binary files. Binary consists of only two digits, 0 or 1.

Read

Write

Append

- **Pickling and Unpickling:**
 - Pickling refers to the process of converting structures (such as lists, dictionaries, etc.) to a byte stream before writing to the file.
 - Unpickling refers to the process of converting a byte stream to structures (such as lists, dictionaries, etc.) before writing to the file.

File Handling

Binary File:

- Python provides a module called the **pickle module**, which includes two functions for reading and writing data to a file.

1. `dump()`: This function is used to dump (write) the content to the file.

Syntax: `f1=pickle.dump(structure, file_object)`

2. `load()`: This function is used to read data from a file.

Syntax: `f1= pickle.load(file_object)`

File Handling

Write a list element in a file

```
import pickle
f=open("binary.dat","wb")
l=["Ram", "Shyam", "Geeta"]
pickle.dump(l,f)
f.close()
print("data successfully inserted")
```

data successfully inserted

Write a list and a dictionary element in a file

```
import pickle
f=open("binary.dat","wb")
l=["Ram", "Shyam", "Geeta"]
d={"Name": "Ram", "Age": 25, "Salary": 25000}
pickle.dump(l,f)
pickle.dump(d,f)
f.close()
print("data successfully inserted")
```

data successfully inserted

Write a dictionary element in a file

```
import pickle
f=open("binary.dat","wb")
d={"Name": "Ram", "Age": 25, "Salary": 25000}
pickle.dump(d,f)
f.close()
print("data successfully inserted")
```

data successfully inserted

File Handling

Read list elements in a file

```
import pickle
f=open("binary.dat","rb")
data=pickle.load(f)
print(data)
f.close()
```

```
['Ram', 'Shyam', 'Geeta']
```

Read list and dictionary elements in a file

```
import pickle
f=open("binary.dat","rb")
data_list=pickle.load(f)
data_dict=pickle.load(f)
print(data_list, data_dict)
f.close()
```

```
['Ram', 'Shyam', 'Geeta'] {'Name': 'Ram', 'Age': 25, 'Salary': 25000}
```

Read dictionary elements in a file

```
import pickle
f=open("binary.dat","rb")
data=pickle.load(f)
print(data)
f.close()
```

```
{'Name': 'Ram', 'Age': 25, 'Salary': 25000}
```

File Handling

Append

```
import pickle
f=open("binary.dat","wb")
l=["Ram", "Shyam", "Geeta"]
pickle.dump(l,f)
f.close()
print("data successfully inserted")
```

data successfully inserted

```
import pickle
f=open("binary.dat","ab")
l=["Raj", "Neha"]
pickle.dump(l,f)
f.close()
print("data successfully appended")
```

data successfully appended

```
import pickle
f=open("binary.dat","rb")
data_list=pickle.load(f)
data_dict=pickle.load(f)
print(data_list, data_dict)
f.close()
```

['Amit', 'Riya', 'Sita'] ['Raj', 'Neha']

File Handling

Write multiple records in a file

```
import pickle
record=[]
f=open("binary.dat","wb")
while True:
    rno=int(input("enter roll no. "))
    name= input("Enter the name")
    marks= int(input("Enter the marks"))
    data=[rno, name, marks]
    record.append(data)
    ch=input("Do you want to add more records? (Y/N): ")
    if ch=="N" or ch=="n":
        break
pickle.dump(record,f)
```

```
enter roll no. 1
Enter the name a
Enter the marks 3
Do you want to add more records? (Y/N): y
enter roll no. 2
Enter the name b
Enter the marks 5
Do you want to add more records? (Y/N): n
```

```
import pickle
f=open("binary.dat","rb")
data=pickle.load(f)
print(data)
f.close()
```

```
[[1, 'a', 3], [2, 'b', 5]]
```

```
f=open("binary.dat","rb")
data=pickle.load(f)
for i in data:
    print(i)
f.close()
```

```
[1, 'a', 3]
[2, 'b', 5]
```

File Handling

Append the data in previous example

```
import pickle
record=[]
f=open("binary.dat","wb")
while True:
    rno=int(input("enter roll no. "))
    name= input("Enter the name")
    marks= int(input("Enter the marks"))
    data=[rno, name, marks]
    record.append(data)
    ch=input("Do you want to add more records? (Y/N): ")
    if ch=="N" or ch=="n":
        break
pickle.dump(record,f)
```

```
enter roll no. 1
Enter the name a
Enter the marks 30
Do you want to add more records? (Y/N): y
enter roll no. 2
Enter the name b
Enter the marks 50
Do you want to add more records? (Y/N): n
```

```
f=open("binary.dat","ab")

while True:
    rno=int(input("Enter roll no: "))
    name=input("Enter the name: ")
    marks=int(input("Enter the marks: "))

    data=[rno,name,marks]

    pickle.dump(data,f)

    ch=input("Do you want to add more records? (Y/N): ")
    if ch=="n" or ch=="N":
        break
```

```
Enter roll no: 3
Enter the name: c
Enter the marks: 80
Do you want to add more records? (Y/N): y
Enter roll no: 4
Enter the name: d
Enter the marks: 60
Do you want to add more records? (Y/N): n
```

File Handling

Append the data in previous example

```
import pickle
f=open("binary.dat","rb")
data_1=pickle.load(f)
data_2=pickle.load(f)
data_3=pickle.load(f)
data_4=pickle.load(f)
print(data_1,data_2,data_3,data_4)
f.close()
```

```
[1, 'a', 30] [2, 'b', 50] [3, 'c', 80] [4, 'd', 60]
```

```
f = open("binary.dat","rb")

# find end of file position
f.seek(0,2)
end = f.tell()

# move pointer back to beginning
f.seek(0,0)

while f.tell() < end:
    data = pickle.load(f)
    print(data)

f.close()
```

```
[1, 'a', 30]
[2, 'b', 50]
[3, 'c', 80]
[4, 'd', 60]
```

File Handling

CSV (Comma Separated Values) files:

- CSV format is one of the simplest and most common ways to store tabular data, i.e., in rows and columns.
- Save with .csv file extension.
- When you open this file in MS-Excel, it will be shown in the format of rows and columns, whereas when opening the same file in a text editor via Notepad, the data will be represented with a comma(,) as a separator.
- It is used to handle large amounts of data, which can be imported into any other application.
- The csv module is used to read and write CSV files.

File Handling

Reading from CSV File:

- First import csv module
- Use open() to open a CSV file; it will return a file object.
- Pass the file object to the reader object
- Perform the operation you want

```
import csv
f=open("students.csv","r")
csv_f=csv.reader(f)
for line in csv_f:
    print(line)
```

```
['Roll', 'Name', 'Age', 'Course']
['1', 'Ankit', '12', 'ML']
['2', 'Vijay', '15', 'DL']
['3', 'Vargiya', '13', 'IP']
['4', 'Ram', '11', 'DL']
['5', 'Shyam', '10', 'IP']
['6', 'Geeta', '8', 'ML']
['7', 'Sita', '9', 'SS']
['8', 'Mohan', '12', 'ML']
['9', 'Pyare', '14', 'SS']
['10', 'Mukesh', '15', 'AI']
```

```
with open ("students.csv","r") as f:
    csv_f=csv.reader(f)
    for line in csv_f:
        print(line)
```

```
['Roll', 'Name', 'Age', 'Course']
['1', 'Ankit', '12', 'ML']
['2', 'Vijay', '15', 'DL']
['3', 'Vargiya', '13', 'IP']
['4', 'Ram', '11', 'DL']
['5', 'Shyam', '10', 'IP']
['6', 'Geeta', '8', 'ML']
['7', 'Sita', '9', 'SS']
['8', 'Mohan', '12', 'ML']
['9', 'Pyare', '14', 'SS']
['10', 'Mukesh', '15', 'AI']
```

```
with open ("students.csv","r") as f:
    csv_f=csv.reader(f)
    for line in csv_f:
        print(line[2])
```

```
Age
12
15
13
11
10
8
9
12
14
15
```

File Handling

Reading CSV as a Dictionary:

- Use csv.DictReader()
- Returns data as a dictionary
- Column name becomes Keys

```
with open ("students.csv","r") as f:  
    csv_f=csv.DictReader(f)  
    for line in csv_f:  
        print(line)
```

```
{'Roll': '1', 'Name': 'Ankit', 'Age': '12', 'Course': 'ML'}  
{'Roll': '2', 'Name': 'Vijay', 'Age': '15', 'Course': 'DL'}  
{'Roll': '3', 'Name': 'Vargiya', 'Age': '13', 'Course': 'IP'}  
{'Roll': '4', 'Name': 'Ram', 'Age': '11', 'Course': 'DL'}  
{'Roll': '5', 'Name': 'Shyam', 'Age': '10', 'Course': 'IP'}  
{'Roll': '6', 'Name': 'Geeta', 'Age': '8', 'Course': 'ML'}  
{'Roll': '7', 'Name': 'Sita', 'Age': '9', 'Course': 'SS'}  
{'Roll': '8', 'Name': 'Mohan', 'Age': '12', 'Course': 'ML'}  
{'Roll': '9', 'Name': 'Pyare', 'Age': '14', 'Course': 'SS'}  
{'Roll': '10', 'Name': 'Mukesh', 'Age': '15', 'Course': 'AI'}
```

```
with open ("students.csv","r") as f:  
    csv_f=csv.DictReader(f)  
    for line in csv_f:  
        print(line["Name"])
```

```
Ankit  
Vijay  
Vargiya  
Ram  
Shyam  
Geeta  
Sita  
Mohan  
Pyare  
Mukesh
```

File Handling

Writing in CSV File:

- First, open the file in writing mode “w”
- use csv.writer()
- Writerow() writes one row

```
fw=open("student.csv","w")
csv_f=csv.writer(fw)
csv_f.writerow(['Name','Age'])
csv_f.writerow(['Raj',20])
fw.close()
```

```
with open("student.csv","w") as fw:
    csv_f=csv.writer(fw)
    csv_f.writerow(['Name','Age'])
    csv_f.writerow(['Raj',20])
```

File Handling

Writing CSV as a Dictionary:

- Use csv.DictWriter()
- Provide fieldnames
- Use writerheader()

```
with open("student.csv","w") as fw:
    field=["Name", "Age"]
    writer = csv.DictWriter(fw, fieldnames=field)
    writer.writeheader()
```

```
: import csv

with open("student.csv","w", newline='') as fw:
    field=["Name", "Age"]
    writer = csv.DictWriter(fw, fieldnames=field)
    writer.writeheader()
    n = int(input("Enter number of students: "))
    for i in range(n):
        name = input("Enter name: ")
        age = input("Enter age: ")
        writer.writerow({"Name": name, "Age": age})
```

```
Enter number of students: 2
Enter name: Ankit
Enter age: 22
Enter name: Vijay
Enter age: 25
```

File Handling

Append:

```
import csv
with open("student.csv","a", newline='') as f:
    writer = csv.writer(f)
    n = int(input("Enter number: "))
    for i in range(n):
        name = input("Name: ")
        age = input("Age: ")

        writer.writerow([name, age])
```

```
Enter number: 2
Name: ram
Age: 10
Name: geeta
Age: 12
```

File Handling

File Pointer:

- The file pointer indicates the current position in the file where reading or writing occurs.

tell() function

- It returns the current position of the file pointer.

Syntax:

`file.tell()`

```
f=open("binary.dat","rb")
print(f.tell())
```

0

0 means beginning of the file

```
import pickle

f=open("binary.dat","rb")

print("Before reading:",f.tell())

data=pickle.load(f)

print("After reading:",f.tell())

f.close()
```

Before reading: 0
After reading: 40

File Handling

File Pointer:

seek() function

- It moves the file pointer to a specific position.

Syntax:

```
file.seek(offset, reference)
```

Reference meaning:

0: Beginning of file

1: Current Position

2: End of File

Move from beginning

```
f.seek(10,0)
```

Move 10 bytes from the beginning of the file.

Move from current position

```
f.seek(5,1)
```

Move 5 bytes forward from the current position.

Move from end

```
f.seek(0,2)
```

Move 0 bytes from the end → pointer goes to the end of file (used for append).